

第5章 字符串

5.1 Trie、KMP以及AC自动机

Trie、KMP以及AC自动机是解决字符串相关问题最常见的数据结构。

例5-1 【Trie】背单词 (Remember the Word, UVa 1401)

给出一个由 S ($1 \leq S \leq 4000$) 个不同长度的单词 (每个单词长度不超过100) 组成的字典和一个长度不超过300 000的长字符串。把这个字符串分解成字典中若干个单词的连接 (单词可以重复使用), 有多少种方法? 比如, 字典中有4个单词a, b, cd, ab, 则abcd有两种分解方法: a+b+cd和ab+cd。输出分解方案数除以20 071 207的余数。

【代码实现】

```

// 刘汝佳
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 4000 * 100 + 10, SIGMA = 26;

// 字母表为全体小写字母的 Trie
struct Trie {
    int ch[MAXN][SIGMA], val[MAXN], sz; // 结点总数
    void clear() { sz = 1, fill_n(ch[0], SIGMA, 0); } // 初始时只有一个根结点
    int idx(char c) { return c - 'a'; } // 字符 c 的编号
    // 插入字符串 s, 附加信息为 v. 注意 v 必须非 0, 因为 0 代表“本结点不是单词结点”
    void insert(const char *s, int v) {
        int u = 0, n = strlen(s);
        for (int i = 0; i < n; i++) {
            int c = idx(s[i]);
            if (!ch[u][c]) // 结点不存在, 新建
                fill_n(ch[sz], SIGMA, 0), val[sz] = 0, ch[u][c] = sz++;
            u = ch[u][c]; // 往下走
        }
        val[u] = v; // 字符串最后一个字符的附加信息为 v
    }

    // 查找 s 的长度不超过 len 的前缀
    void find_prefixes(const char *s, int len, vector<int> &ans) {
        int u = 0;
        for (int i = 0; i < len; i++) {
            if (s[i] == '\0') break;
            int c = idx(s[i]);
            if (!ch[u][c]) break;
            u = ch[u][c];
            if (val[u] != 0) ans.push_back(val[u]); // 找到一个前缀
        }
    }
};

// 文本串最大长度, 单词最大个数
const int TL = 3e5 + 4, WC = 4000 + 4, MOD = 20071027;
int D[TL], WLen[WC];
Trie trie;

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    string text, word;
    for (int kase = 1; S; cin >> text >> S; kase++) {
        trie.clear();
        for (int i = 1; i <= S; i++)
            cin >> word, WLen[i] = word.length(), trie.insert(word.c_str(), i);
        int L = text.length();
        fill_n(D, L, 0), D[L] = 1;
        for (int i = L - 1; i >= 0; i--) {
            vector<int> p;
            trie.find_prefixes(text.c_str() + i, L - i, p);
            for (size_t j = 0; j < p.size(); j++)
                (D[i] += D[i + WLen[p[j]]]) %= MOD;
        }
        printf("Case %d: %d\n", kase, D[0]);
    }
    return 0;
}

/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 3.3.1 节例题 13
同类题目: "strcmp()" Anyone?, Uva 11732
Dictionary Size, ACM/ICPC NEERC 2011, Uva 1519
Xor Sum, HDU 4825
Chip Factory, HDU 5536
*/

```

例5-2 【KMP；失配函数】周期（Period, SEERC 2004, POJ 1961)

给定一个长度为 n ($2 \leq n \leq 10^6$) 的字符串 S ，求它每个前缀的最短循环节。换句话说，对于每个 i ($2 \leq i \leq n$)，求一个最大的整数 $K > 1$ （如果 K 存在），使得 S 的前 i 个字符组成的前缀是某个字符串重复 K 次得到的。输出所有存在 K 的 i 和对应的 K 。

比如，对于字符串aabaabaabaab，只有当 $i=2, 6, 9, 12$ 时 K 存在，且分别为2, 2, 3, 4。

【代码实现】

```
// 刘汝佳
#include <cstdio>
const int NN = 1e6 + 4;
char P[NN];
int F[NN];

int main() {
    for (int n, kase = 1; scanf("%d", &n) == 1 && n; kase++) {
        scanf("%s", P);
        F[0] = 0, F[1] = 0; // 递推边界初值
        for (int i = 1; i < n; i++) { // 递推计算失配函数
            int j = F[i];
            while (j && P[i] != P[j]) j = F[j];
            F[i + 1] = (P[i] == P[j] ? j + 1 : 0);
        }
        printf("Test case #%d\n", kase);
        for (int i = 2; i <= n; i++)
            if (F[i] > 0 && i % (i - F[i]) == 0) printf("%d %d\n", i, i / (i - F[i]));
        printf("\n");
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 3.3.2 节例题 15
同类题目：Cyclic Nacklace, HDU 3746
          Number Sequence, HDU 1711
          Seek the Name, Seek the Fame, POJ 2752
          Power Strings, POJ 2406
          Simpsons' Hidden Talents, HDU 2594
*/
```

例5-3 【AC自动机】出现次数最多的子串 (Dominating Patterns, UVa 1449)

有 n ($1 \leq n \leq 150$) 个由小写字母组成的字符串和一个文本串 T , 长度为 $1 \sim 10^6$ 的整数。找出哪些字符串在文本中出现的次数最多。例如, 字符串aba在ababa中出现2次, 但字符串bab只出现了1次。

对于每组数据, 第一行输出最多出现的次数, 接下来每行包含一个出现次数最多的字符串, 按照输入顺序排列。

【代码实现】

```
// 刘汝佳
#include <bits/stdc++.h>
using namespace std;

const int SIGMA = 26, MAXN = 11000, MAXS = 150 + 10;
map<string, int> ms;
struct AhoCorasickAutomata {
    int ch[MAXN][SIGMA];
    int f[MAXN]; // fail 函数
    int val[MAXN]; // 每个字符串的结尾结点都有一个非 0 的 val
    int last[MAXN]; // 输出链表的下一个结点
    int cnt[MAXS], sz;
    void init() {
```

```

    sz = 1, fill_n(ch[0], SIGMA, 0), fill_n(cnt, MAXS, 0), ms.clear();
}
int idx(char c) { return c - 'a'; } // 字符 c 的编号
void insert(char* s, int v) { // 插入字符串, v 必须非 0
    int u = 0, n = strlen(s);
    for (int i = 0; i < n; i++) {
        int c = idx(s[i]);
        if (!ch[u][c]) fill_n(ch[sz], SIGMA, 0), val[sz] = 0, ch[u][c] = sz++;
        u = ch[u][c];
    }
    val[u] = v, ms[string(s)] = v;
}

// 递归打印以结点 j 结尾的所有字符串
void print(int j) {
    if (j) cnt[val[j]]++, print(last[j]);
}

// 在 T 中找模板
void find(char* T) {
    int n = strlen(T), j = 0; // 当前结点编号, 初始为根结点
    for (int i = 0; i < n; i++) { // 文本串当前指针
        int c = idx(T[i]);
        while (j && !ch[j][c]) j = f[j]; // 顺着细边走, 直到可以匹配
        j = ch[j][c];
        if (val[j])
            print(j);
        else if (last[j])
            print(last[j]); // 找到了
    }
}

// 计算 fail 函数
void getFail() {
    queue<int> q;
    f[0] = 0;
    for (int c = 0; c < SIGMA; c++) { // 初始化队列
        int u = ch[0][c];
        if (u) f[u] = 0, q.push(u), last[u] = 0;
    }
    while (!q.empty()) { // 按 BFS 顺序计算 fail
        int r = q.front();
        q.pop();
        for (int c = 0; c < SIGMA; c++) {
            int u = ch[r][c];
            if (!u) continue;
            q.push(u);
            int v = f[r];
            while (v && !ch[v][c]) v = f[v];

```

```

        f[u] = ch[v][c], last[u] = val[f[u]] ? f[u] : last[f[u]];
    }
}
};

AhoCorasickAutomata ac;

char text[1000001], P[MAXS][80];
int main() {
    for (int n; scanf("%d", &n) == 1 && n;) {
        ac.init();
        for (int i = 1; i <= n; i++) scanf("%s", P[i]), ac.insert(P[i], i);
        ac.getFail();
        scanf("%s", text), ac.find(text);
        int best = *max_element(ac.cnt + 1, ac.cnt + n + 1);
        printf("%d\n", best);
        for (int i = 1; i <= n; i++)
            if (ac.cnt[ms[string(P[i])]] == best) printf("%s\n", P[i]);
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 3.3.3 节例题 16
同类题目：GRE Words Revenge, ACM/ICPC Chengdu 2013, UVa 1676
          First of Her Name, ACM/ICPC WF 2019, Codeforces Gym 102511G
*/

```

例5-4 【AC自动机】子串 (Substring, UVa 11468)

给出一些字符和各自对应的选择概率，随机选择 L 次后将得到一个长度为 L 的随机字符串 S （每次独立随机）。给出 K ($K \leq 20$) 个模板串，计算 S 不包含任何一个串的概率（即任何一个模板串都不是 S 的连续子串）。

【代码实现】

```

// 刘汝佳/陈锋
#include <bits/stdc++.h>
using namespace std;
const int SIGMA = 64, MAXN = 500, MAXS = 20 + 10; // 结点总数, 模板个数
int idx[256], n;
double prob[SIGMA];
struct AhoCorasickAutomata {
    int ch[MAXN][SIGMA];
    int f[MAXN]; // fail 函数
    int match[MAXN]; // 是否包含某个字符串
    int sz; // 结点总数
    void init() { sz = 1, fill_n(ch[0], SIGMA, 0); }
    void insert(const char *s) { // 插入字符串
        int u = 0, n = strlen(s);

```

```

    for (int i = 0; i < n; i++) {
        int c = idx[s[i]];
        if (!ch[u][c]) fill_n(ch[sz], SIGMA, 0), match[sz] = 0, ch[u][c] = sz++;
        u = ch[u][c];
    }
    match[u] = 1;
}

void getFail() { // 计算 fail 函数
    queue<int> q;
    f[0] = 0;
    for (int c = 0; c < SIGMA; c++) { // 初始化队列
        int u = ch[0][c];
        if (u) f[u] = 0, q.push(u);
    }
    while (!q.empty()) { // 按 BFS 顺序计算 fail
        int r = q.front();
        q.pop();
        for (int c = 0; c < SIGMA; c++) {
            int u = ch[r][c];
            if (!u) {
                ch[r][c] = ch[f[r]][c];
                continue;
            }
            q.push(u);
            int v = f[r];
            while (v && !ch[v][c]) v = f[v];
            f[u] = ch[v][c], match[u] |= match[f[u]];
        }
    }
}

void dump() {
    printf("sz = %d\n", sz);
    for (int i = 0; i < sz; i++)
        printf("%d: %d %d %d\n", i, ch[i][0], ch[i][1], match[i]);
    printf("\n");
}

};

AhoCorasickAutomata ac;

double d[MAXN][105];
int vis[MAXN][105];
double getProb(int u, int L) {
    if (!L) return 1.0;
    if (vis[u][L]) return d[u][L];
    vis[u][L] = 1;
    double &ans = d[u][L];

```

```

    ans = 0.0;
    for (int i = 0; i < n; i++)
        if (!ac.match[ac.ch[u][i]]) ans += prob[i] * getProb(ac.ch[u][i], L - 1);
    return ans;
}

char s[30][30];

int main() {
    int T;
    scanf("%d", &T);
    for (int kase = 1, k, L; kase <= T; kase++) {
        scanf("%d", &k);
        for (int i = 0; i < k; i++) scanf("%s", s[i]);
        scanf("%d", &n);
        for (int i = 0; i < n; i++) {
            char ch[9];
            scanf("%s%lf", ch, &prob[i]), idx[ch[0]] = i;
        }
        ac.init();
        for (int i = 0; i < k; i++) ac.insert(s[i]);
        ac.getFail();
        scanf("%d", &L);
        memset(vis, 0, sizeof(vis));
        printf("Case #d: %.6lf\n", kase, getProb(0, L));
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 3.3.3 节例题 17
同类题目：DNA Repair, HDU 2457
           Wireless Password, HDU 2825
           Password Suspects, ACM/ICPC, WF 2008, UVa 1076
           Censored!, POJ 1625
*/

```

本节例题列表

本节讲解的例题及其囊括的知识点，如表5-1所示。

表5-1 Trie、KMP以及AC自动机例题归纳